SOC ANALYST PORTFOLIO CASE STUDY

Cowrie Honeypot Detection and SIEM Monitoring

Building a non-root SSH deception service, collecting JSON telemetry, and mapping custom Wazuh detections to MITRE ATT&CK

One controlled SSH session produced 17 raw Cowrie events and 8 validated Wazuh alerts across authentication and command execution.

Analyst	Ryan Bryant
Exercise date	August 26, 2026
Environment	Authorized VirtualBox SOC home lab
Classification	Portfolio demonstration - no production systems

AUTHORIZED-LAB SAFETY NOTE

All scanning, authentication attempts, firewall actions, and log collection occurred inside a controlled private network owned and operated by the analyst.

Executive Summary

The analyst deployed Cowrie on LINUX-WEBSERVER01 as a dedicated non-root service listening on TCP 2222, restricted exposure to the private 10.10.10.0/24 lab network, and configured the Wazuh agent to ingest Cowrie's JSON audit log. Three custom Wazuh rules detected failed authentication, successful authentication, and interactive command input. A controlled Parrot session generated one failed login, one successful simulated login, and six command events. Cowrie preserved 17 raw session records, Wazuh generated 8 correlated alerts, and all 8 collected evidence files passed SHA-256 verification. The interaction remained inside Cowrie's emulated environment and did not expose the real Ubuntu filesystem.

DISPOSITION

End-to-end deception telemetry and SIEM detection validated. No real host compromise or production exposure occurred.

Objective and Scope

- **Primary objective:** Deploy an SSH honeypot and prove that authentication and command activity reaches the SIEM.
- **Collection scope:** Cowrie JSON events from a single controlled SSH session on TCP 2222.
- **Detection scope:** Custom Wazuh rules for failed login, successful login, and Unix shell command input.
- **Safety scope:** Non-root service account, isolated lab-only firewall exposure, and no redirection of the real SSH service on TCP 22.

Lab Environment

Role	System	Address / Function
Client	Parrot Security	10.10.10.5 - controlled SSH interaction
Honeypot endpoint	LINUX-WEBSERVER01	10.10.10.4:2222 - Cowrie and Wazuh agent
SIEM	Wazuh server	10.10.10.3 - JSON decoding and alert analysis
Network	VirtualBox NAT Network	10.10.10.0/24 - isolated lab segment

Secure Deployment Design

- Created a dedicated cowrie account, locked password login, and ran the honeypot outside root context.
- Installed Cowrie in a Python virtual environment under /home/cowrie/my-honeypot.

- Kept the real SSH service on TCP 22 unchanged and used Cowrie's default SSH honeypot port, TCP 2222.
- Allowed TCP 2222 only from 10.10.10.0/24 with UFW.
- Created a simulated lab credential in Cowrie userdb; no corresponding Linux account was created.

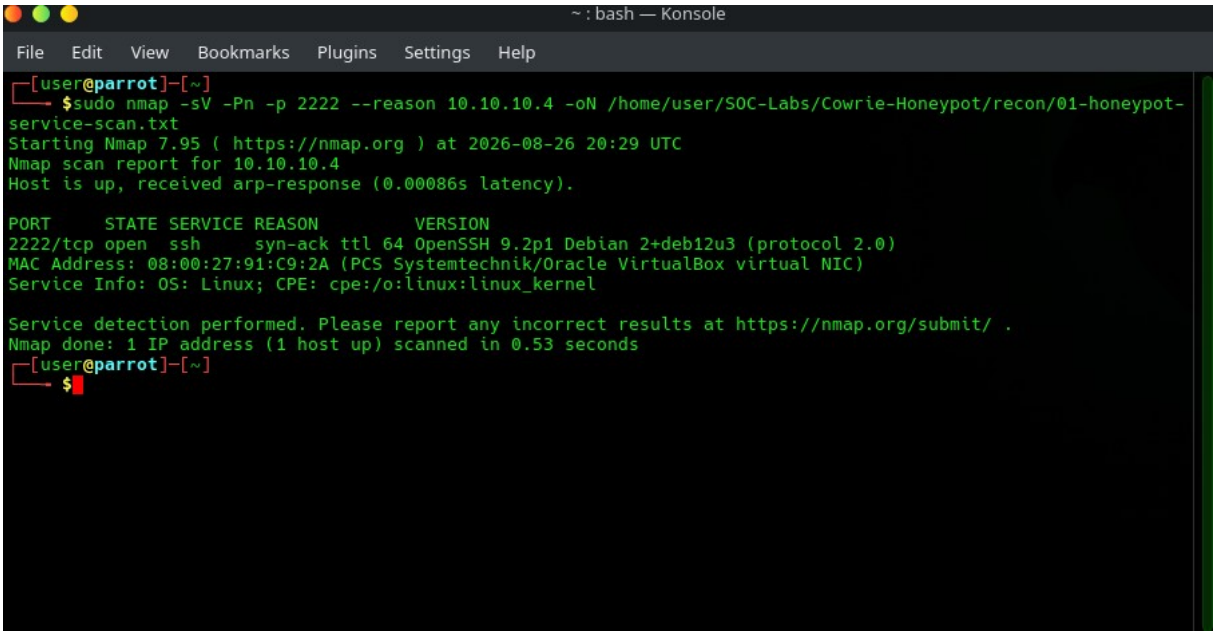

Figure 1. Nmap identified an SSH service on TCP 2222, confirming the honeypot was reachable only on the intended lab port.

Wazuh Log Collection and Detection Engineering

The Wazuh agent was configured with a localfile entry using log_format json and the Cowrie JSON path. The collector configuration was validated before the agent restart. On the manager, three custom rules were added and tested before production use in the lab.

Rule	Level	Cowrie event	MITRE ATT&CK context
100200	7	cowrie.login.failed	T1110 - Brute Force
100201	9	cowrie.login.success	T1078 - Valid Accounts
100202	10	cowrie.command.input	T1059.004 - Unix Shell

ATT&CK mappings describe the behavior category represented by the telemetry; they do not by themselves prove a real adversary or compromise.

Controlled Interaction

The analyst recorded a controlled SSH session from Parrot. The first password attempt intentionally failed; the second used the simulated Cowrie credential. Inside the emulated shell, the analyst entered whoami, uname -a, pwd, ls -la, cat /etc/passwd, and exit.

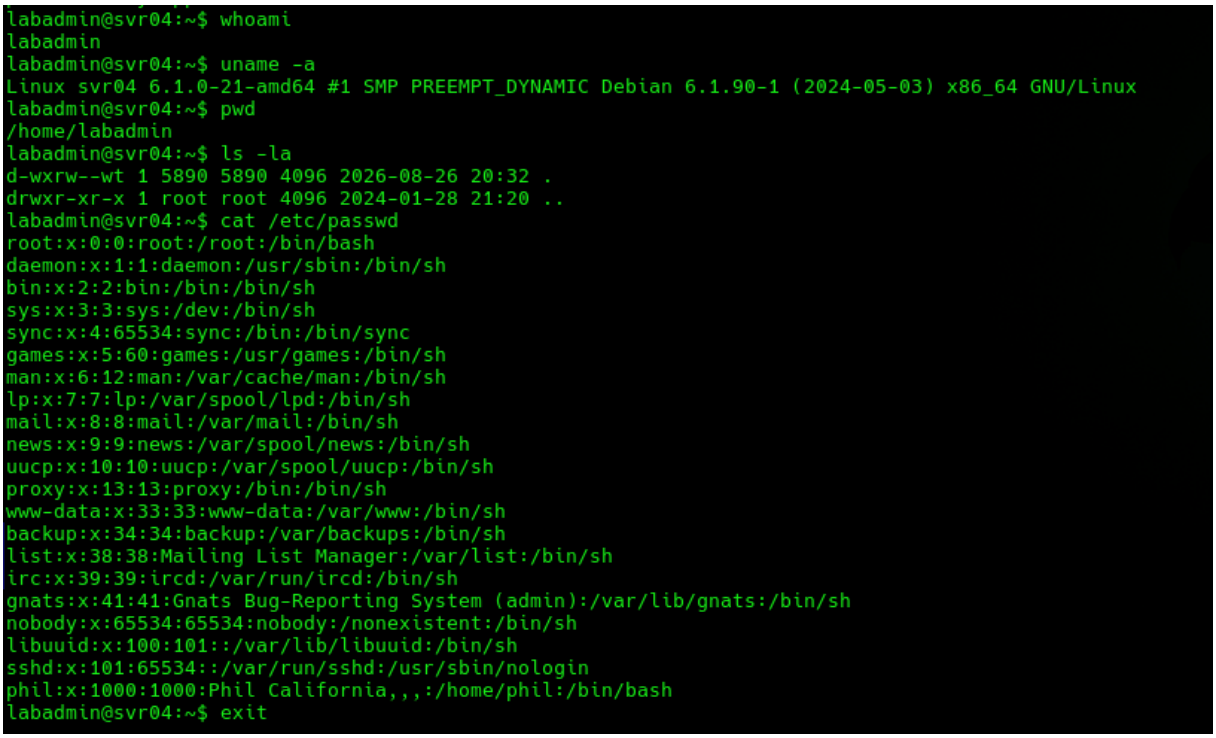

Figure 2. Cowrie returned an emulated Linux identity, kernel, directory listing, and passwd file; the real Ubuntu filesystem was not exposed.

Detection Results

VALIDATED ALERT COUNT

8 Wazuh alerts: 1 failed login + 1 successful login + 6 command-input events

The raw Cowrie log contained 17 records for session 5a4b944ac7ac. Wazuh created alerts only for the three event classes targeted by the custom rules, producing the expected 8-event sequence.

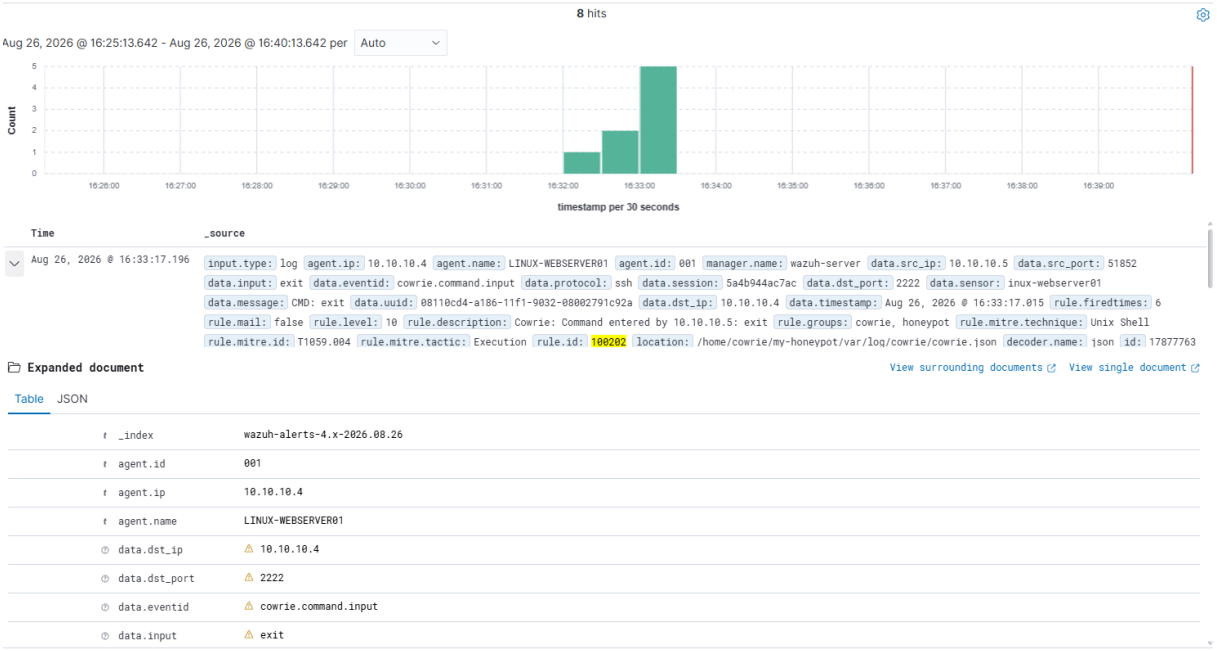

Figure 3. Wazuh Discover showed eight custom-rule hits for the controlled Cowrie session.

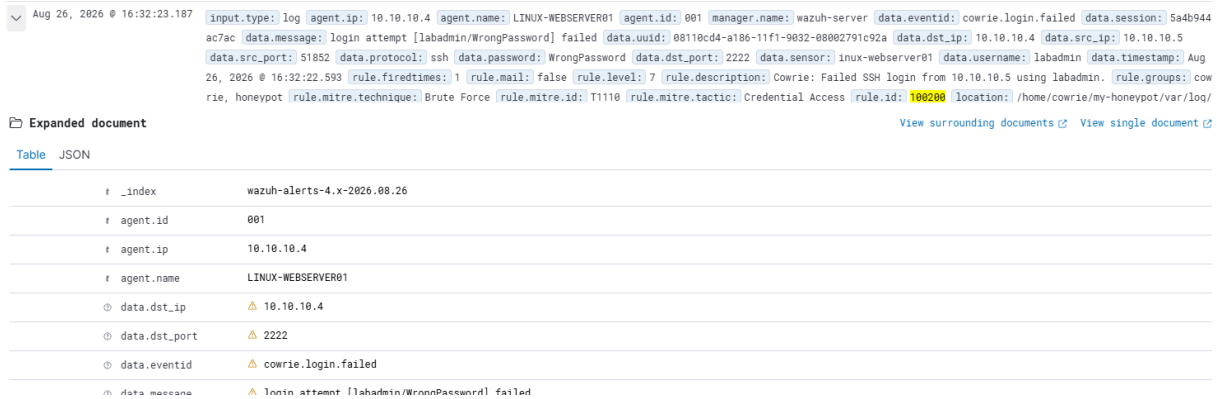

Figure 4. Rule 100200 detected the failed SSH login at level 7 and mapped the event to T1110 Brute Force.

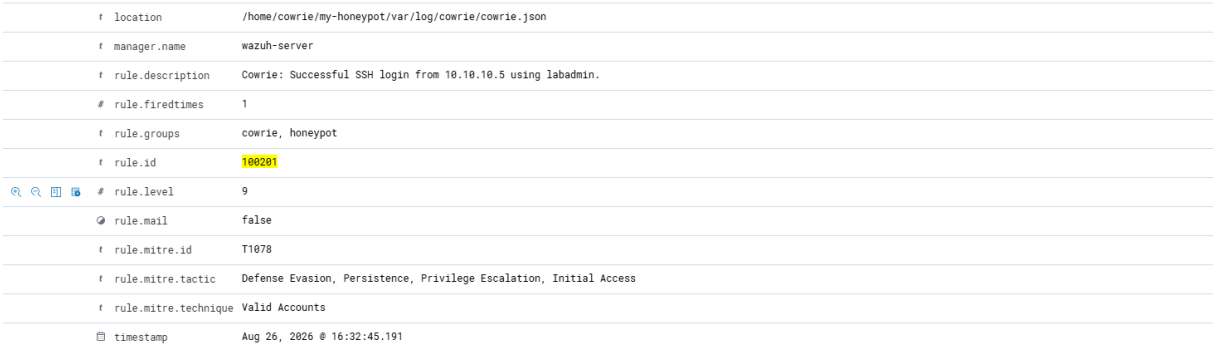

Figure 5. Rule 100201 detected the successful simulated login at level 9 and mapped it to T1078 Valid Accounts.

	manager.name	wazuh-server
	rule.description	Cowrie: Command entered by 10.10.10.5: exit
	rule.firedtimes	6
	rule.groups	cowrie, honeypot
	rule.id	100202
	rule.level	10
	rule.mail	false
	rule.mitre.id	T1059.004
	rule.mitre.tactic	Execution
	rule.mitre.technique	Unix Shell
	timestamp	Aug 26, 2026 @ 16:33:17.196

Figure 6. Rule 100202 recorded six command-input alerts at level 10 and mapped them to T1059.004 Unix Shell.

Evidence Integrity

The final evidence package contained eight files: the service scan, recorded SSH session, Cowrie session telemetry, Cowrie service status, Wazuh alert export, Wazuh local rules, and start/end timestamps. SHA-256 verification returned OK for every artifact.

```
┌──[user@parrot]─[~/SOC-Labs/Cowrie-Honeypot]
└─── $sha256sum -c evidence/SHA256SUMS.txt
./evidence/02-cowrie-ssh-session.txt: OK
./evidence/cowrie-local-rules.xml: OK
./evidence/cowrie-service-status.txt: OK
./evidence/cowrie-session-events.jsonl: OK
./evidence/cowrie-wazuh-alerts.jsonl: OK
./notes/lab-end.txt: OK
./notes/lab-start.txt: OK
./recon/01-honeypot-service-scan.txt: OK
┌──[user@parrot]─[~/SOC-Labs/Cowrie-Honeypot]
└─── $
```

Figure 7. All eight Cowrie Honeypot evidence files passed SHA-256 verification.

Evidence-to-Conclusion Matrix

Evidence	Observed fact	Analyst conclusion
Nmap service scan	TCP 2222 open as SSH	Cowrie was reachable on the intended deception port
Session transcript	Emulated shell returned controlled output	Interaction remained inside the honeypot experience
Cowrie JSON	17 records for one session	Raw telemetry preserved connection, authentication, and activity context

Evidence	Observed fact	Analyst conclusion
Wazuh alerts	8 alerts across three custom rules	End-to-end collection, decoding, and detection worked as designed
Rule metadata	Levels 7, 9, and 10 with ATT&CK IDs	Authentication and command activity received differentiated severity and context
Hash verification	8 files reported OK	Collected artifacts were unchanged after hashing

Findings and Limitations

Confirmed findings

- Cowrie successfully emulated SSH on TCP 2222 under a dedicated non-root account.
- Wazuh collected the JSON log from the endpoint and decoded Cowrie fields including session, source, destination, username, event ID, and command input.
- The three custom rules generated the expected severities and ATT&CK context for the controlled activity.
- The session exposed only the honeypot's emulated environment; no evidence showed access to the real Ubuntu filesystem.

What was not proven

- The lab did not test internet exposure, real adversary infrastructure, malware delivery, persistence, or outbound command-and-control activity.
- A single scripted session does not establish enterprise-scale performance, false-positive rates, or alert tuning thresholds.
- ATT&CK mappings are analytic context; they are not attribution and do not prove malicious intent outside the controlled scenario.

Recommendations

Priority	Recommendation	Rationale
High	Place production honeypots in a dedicated monitored segment with strict outbound controls.	Limits risk if a deception service is abused and improves traffic attribution.
High	Restrict access to captured credentials and command logs as sensitive security telemetry.	Honeypots intentionally collect material that may include real secrets.
High	Correlate authentication success with command activity and session identifiers.	Raises confidence and preserves the behavior sequence across multiple alerts.

Priority	Recommendation	Rationale
Medium	Add rate-based rules and source reputation enrichment.	Separates one-off probes from sustained credential attacks.
Medium	Monitor honeypot process health, port status, and log-collection gaps.	Prevents silent monitoring failure from being mistaken for an absence of activity.

Troubleshooting and Engineering Judgment

- Backed up Wazuh configuration and rule files before editing and validated syntax before restarting services.
- Recovered from malformed XML quotation marks by restoring the verified backup, then used a quote-safe encoded append method.
- Validated rule 100200 with wazuh-logtest before restarting the manager, preventing an untested configuration from reaching the live lab.
- Used a FIPS-approved ECDH key exchange when the Wazuh server rejected Curve25519 during evidence transfer.
- Stopped Parrot's temporary SSH service after collection and shut down all VMs through their operating systems rather than saving frozen states.

Employer-Facing Project Summary

This project demonstrates detection engineering from collection through validation: deploy a safe deception service, configure structured JSON ingestion, write and test custom SIEM rules, generate controlled telemetry, correlate authentication and command activity, and preserve a verifiable evidence package.

Resume Bullet

PORTFOLIO-READY BULLET

Deployed a non-root Cowrie SSH honeypot integrated with Wazuh JSON collection; engineered and validated three custom detections that generated 8 alerts from 17 raw session events and preserved 8 SHA-256-verified evidence artifacts.

Interview Talking Points

- Why Cowrie remained on TCP 2222 instead of replacing or redirecting the real SSH service.
- How session identifiers linked the failed login, successful login, and command sequence into one coherent activity chain.

- Why rule validation and backup restoration mattered before restarting the SIEM manager.

Skills Demonstrated

Cowrie honeypot | Wazuh log collection | JSON decoding | Custom SIEM rules | MITRE ATT&CK mapping | SSH telemetry | Detection validation | Evidence integrity

AI and Source Transparency

The analyst personally configured the virtual machines, executed the commands, validated the alerts, interpreted the results, captured the screenshots, and preserved the evidence. Generative AI assisted with command sequencing, troubleshooting suggestions, and documentation structure. Technical claims were checked against direct lab evidence and the official references listed below.

References

- Cowrie documentation - Installing Cowrie. https://docs.cowrie.org/en/latest/INSTALL.html
- Wazuh documentation - Localfile configuration. https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/localfile.html
- Wazuh documentation - Custom rules. https://documentation.wazuh.com/current/user-manual/ruleset/rules/custom.html
- MITRE ATT&CK - Brute Force T1110. https://attack.mitre.org/techniques/T1110/
- MITRE ATT&CK - Valid Accounts T1078. https://attack.mitre.org/techniques/T1078/
- MITRE ATT&CK - Unix Shell T1059.004. https://attack.mitre.org/techniques/T1059/004/